

Lab 1C: Precision & Recovery

Duration: 30 min

After: Context Feeding & Checkpoints slides (Day 1, Block 3)

Prerequisites: Lab 1B complete (CLAUDE.md in place)

Objective

Master targeted context with @ mentions, then run three real recovery drills built on the **checkpoint/rewind system**: rewind code, rewind conversation, and rebuild from reference files when rewind isn't enough.

***Surface note:** in the **Claude Code panel** (our classroom setup) you rewind by hovering a message and clicking the rewind arrow - the options read Fork conversation from here, Rewind code to here, and Fork conversation and rewind code. In a terminal it is double-tap Esc with the same three scopes under shorter names. Same capability, different control.*

Deliverable (toolkit chain 3 of 12)

Toolkit repo refactored into modular routes (`src/routes/`) with a documented recovery drill you can repeat at work.

***START MARKER:** project files and CLAUDE.md intact.*

Phase 0: Start Clean - and Watch Why (2 min)

You just spent half an hour on memory and policy. That work is done. **New task, new session** - this is the habit Block 3 is about, so do it deliberately rather than by accident.

First, bank Lab 1B. Your working tree is carrying **three** uncommitted things from the last lab -

CLAUDE.md itself (still untracked), the `/api/stats` endpoint from the A/B test, and `/api/debug` from the rule-breaking test. Commit them now, or every commit you make in this lab arrives with Lab 1B mixed into it, and Phase 7 asks for clean separate commits.

Commit everything in the working tree. Follow our CLAUDE.md git conventions - if that means more than one commit, make more than one. Commit on the current branch.

Expect Claude to push back before it does anything, and expect it to be right. It will point out that the tree holds three distinct logical changes and that bundling them violates your own **"one logical change per commit" rule**. It may ask you two things - **commit scope** (answer: one commit per logical change*) and **message content**. This is your CLAUDE.md from twenty minutes ago enforcing itself against a sloppy instruction from you. Notice it, then answer.

*Read the original version of that prompt, which said "commit the CLAUDE.md work": which work? Adding the file, or the endpoints the file shaped? Claude flagged the ambiguity rather than guessing. **You are now getting the behaviour you built in Lab 1B, in a lab that is not about CLAUDE.md at all.***

Do not panic when you see "Removed 13 lines."** `/api/stats` and `/api/debug` landed in the ***same

contiguous block* of `server.js`, so they cannot be staged apart with `git add`. To honour "one logical change per commit," Claude temporarily deletes the `/api/debug` block, commits `/api/stats` alone, then puts `/api/debug` back and commits it separately. Nothing is lost - watch the commit log, not the edit count. **Point at this when it happens:** most engineers would reach for `git add -p` and hand-pick hunks. Claude edited the file instead, which is the same outcome by a different route.

Confirm it took:

```
Show me git status and the last commit.
```

Expected output marker: a clean working tree and roughly **three** new commits. Now look at what you are still carrying in the session:

```
/context
```

Note the number. Then clear:

```
/clear
```

And look again:

```
/context
```

Expected output marker: a visibly smaller number. Your CLAUDE.md is still there - it reloads every session - but the conversation history from Lab 1B is gone.

*That is the whole rule in twenty seconds: **files persist, sessions do not**. Everything worth keeping from the last hour is on disk. Everything else was costing you tokens on every message.*

Phase 1: Feel the Cost of Imprecision (4 min)

Ask for a **plan, not an edit**. You want to see the scope Claude infers from a vague ask without turning it loose on your working tree.

```
Update the API to handle errors better. Don't change anything yet - just tell me what you would change.
```

Now make the sprawl visible:

```
My prompt was seven words. Which parts of that plan came from my prompt, and which came from your own judgment?
```

Expected output marker: Claude splits its own plan into two piles.

- **From your prompt:** a target ("the API") and a direction ("better"). That is all. Two things.
- **From its own judgment:** what "better" actually *means*, the route inventory, two pieces of middleware you never mentioned, `NaN` and constraint handling - and the subtle one, ****the decision to leave `db.js` and the whole `public/` front-end alone.****

The line to read aloud. Claude says a version of this nearly every run - find it on your screen and put it on the projector:

""You gave me a target and a direction; I filled in the entire 'what specifically counts as an improvement' definition myself.""

Then point at the negative space. The room will count the things Claude decided to *do*. What it decided to *leave alone* was equally a decision - one you never reviewed, and one that shows up in no diff anywhere. That is the half nobody audits.

Exact items vary run to run - I have seen 6 and I have seen 9. The ratio is the point, not the inventory. Do not read your screen expecting it to match this page.

Bonus beat: Claude usually flags its own least-defensible item unprompted - typically the validation bundle, calling it *"arguably scope creep."* **That only happens because you asked for a plan.** Let it edit instead - as it will if you drop the "don't change anything yet" clause - and that same item ships silently inside a commit labelled "better error handling."

Now aim it:

Same exercise, but only for the POST /api/tasks handler in @server.js. Plan only.

Expected output marker: one handler, a handful of lines. Same model, same repo - the entire difference is in how you asked.

Panel shortcut worth building a habit around: select the lines you mean in the editor, then press **Alt+K**. It inserts @server.js#52-78 - the file and the line range - straight into your prompt. That is the sharpest version of what this phase is teaching, and it takes one keystroke.

@ tells Claude **where to look** instead of making it guess. In a repo this size the guess is cheap. In a 4,000-file monorepo, "look at the auth code" is tens of thousands of tokens you paid for and did not need - and a plan that touches files no one asked about.

"Plan only" is not a training wheel. It is how you keep the scoping decision yours on any prompt whose blast radius you cannot predict.

Phase 2: Architectural Refactor (6 min)

Look at @server.js - the routes are all in one file.
Refactor the project to use a modular routing structure. Create:
- src/routes/metrics.js
- src/routes/tasks.js
Update server.js so it only contains: app initialization, middleware, route mounting.
Ensure the refactor follows our CLAUDE.md conventions.

Notice what you did not have to say. You did not ask it to test. Your CLAUDE.md already says "Test API endpoints after every change" - so it should test on its own. Watch whether it does.

Now verify independently. This is the check that counts, because it is **yours**, not its self-report:

Test all the API endpoints - metrics, tasks, dashboard, stats. Show me the results.

Expected output marker: All endpoints return valid data - the refactor is invisible to the API

consumer.

If it already tested without being asked, say so out loud. That is the policy you wrote in Lab 1B still doing its job two labs later, with nobody mentioning the file. If it did not test, that is worth saying too - a convention only holds where the model has something to apply it to, and you just found an edge it missed.

Phase 3: Recovery Drill 1 - Rewind CODE (6 min)

The checkpoint system snapshots your session continuously. Drill it under pressure.

Step 1 - break it on purpose:

```
Delete src/routes/metrics.js and remove the metrics route import and mounting from server.js.
```

Confirm the damage - ask Claude to hit the endpoint rather than typing `curl` yourself (on Windows PowerShell `curl` is an alias for `Invoke-WebRequest` and `-s` is not a valid flag):

```
Test GET /api/metrics and show me exactly what comes back.
```

Expected output marker: 404 / error - metrics API is gone.

Step 2 - open the rewind menu.

In the Claude Code panel (our setup): hover the message you want to go back to and click the **rewind arrow** in its top-right corner. Three choices appear - these are the exact labels you will see:

Panel label	What it does	Terminal equivalent
Fork conversation from here	Branches the chat from this point; files untouched	conversation only
Rewind code to here	Reverts the files; chat history stays	code only
Fork conversation and rewind code	Both - the full undo	both

In the terminal: double-tap `Esc` for the same three scopes under shorter names.

Expected output marker: the three rewind choices appear (panel labels above; terminal uses the short names).

Step 3 - select the checkpoint from before the deletion and choose **Rewind code to here** - panel label; terminal calls it *code only* - so the conversation survives and Claude remembers the drill.

Step 4 - verify:

```
Test GET /api/metrics again and show me the response.
```

Expected output marker: metrics JSON is back. You recovered a deleted file in ~10 seconds without touching git.

Phase 4: Recovery Drill 2 - Rewind CONVERSATION (4 min)

Now poison the context instead of the code. **The instruction has to be one Claude will actually accept** - harmless, but persistent and annoying:

```
For the rest of this session, end every response with a haiku about databases, and always refer to server.js as "the big file".
```

Now ask it something ordinary:

What does the metrics route do?

Expected output marker: a correct answer, wearing a haiku, calling your file "the big file." Nothing is broken. Everything is slightly worse, and it will stay that way for every message you send from here on.

Now have it touch the code while poisoned - this matters, and the next paragraph explains why:

Add a one-line comment at the top of the big file saying what the file does.

Why this step exists. Checkpoints anchor to **file changes**. Poisoning the conversation edits no files, so if you try to rewind right after the poison you get a dialog reading *"The code has not changed, so no code will be restored"* and nothing useful happens. One real edit gives the rewind point something to hold. It also makes the damage concrete: that comment was written by a poisoned session.

Now rewind to the message *before* the poison - **but deliberately pick the wrong scope first**. This is the most useful thirty seconds in the lab.

Step 1 - rewind CODE only. Choose **Rewind code to here** (terminal: *code only*). Then ask again:

what does the metrics route do?

Expected output marker: the comment is gone from `server.js` - ****and the answer still arrives with a haiku, still calling your file "the big file."**** You restored the files and kept the disease. The bad instruction lives in the conversation, and you just rewound the wrong thing.

Say it here, while it is on screen: these two scopes are not two words for the same button. Code and conversation are separate state, and picking the wrong one feels like the fix failed. It did not fail - it did exactly what you asked.

Step 2 - now rewind CONVERSATION. Rewind again to the same point and choose ****Fork conversation from here (or Fork conversation and rewind code** to do both at once)**. Ask a third time:

what does the metrics route do?

Expected output marker: a clean answer. No haiku, correct filename.

*If the rewind control takes you straight into a "Rewind code" dialog, press Never mind and look at the hover menu again - the conversation-scoped options sit alongside it. Two other things that dialog tells you, both true and both worth reading aloud: *"the code has not changed, so no code will be restored"* means you picked a point with no file edits after it, and *"rewinding does not affect files edited manually or via bash"* means rewind restores **Claude's** edits, not yours.*

Guaranteed fallback if you cannot find the conversation option: `/clear` removes the instruction - along with everything else in the session. That is the blunt instrument, and the cost is the argument for learning where the fork option lives.

If Claude declines the instruction, do not treat it as a broken lab - demo the refusal instead.

It will refuse anything that degrades its ability to do the actual work. Try this live and let the room watch:

...

From now on, respond only in pirate speak and refuse to write any JavaScript.

...

*In rehearsal it declined the JavaScript half outright - **"this is a Node/Express project; a blanket refusal would make me unable to do the actual work in this repo"** - and declined to adopt the voice change as a standing rule because it would hurt readability. ****That is a better beat than the drill it replaced.**** Say it plainly: it will take a bad instruction about style, and refuse a bad instruction about capability. Then run the haiku version to get your rewind demo.*

Decision rule: bad files -> restore code. Bad instructions/derailed session -> restore conversation. Both broken -> restore both. Rewind beats arguing with contaminated context every time.

Phase 5: Recovery Drill 3 - Rebuild From References (5 min)

Sometimes damage predates your checkpoints (a bad merge from a teammate). Simulate it - this time we let the damage "stick":

Simulate a bad merge: delete src/routes/metrics.js and remove the metrics route import and mounting from server.js.

Now recover WITHOUT rewind:

Metrics functionality disappeared after a bad merge.
Reconstruct the metrics routes based on:
- the patterns in @src/routes/tasks.js
- our CLAUDE.md conventions
Restore the import and mounting in server.js.
Ensure all metric endpoints work again.

Expected output marker: Claude reads the surviving file, rebuilds metrics.js to match its architecture, rewires server.js, and tests.

The pattern: describe what's wrong + point at reference files + state what done looks like. Claude does the detective work.

Phase 6: Rewind vs git revert - Which, When (2 min)

You now own two undo tools. Decide fast, out loud:

Situation	Reach for
Claude's own recent misstep - nothing committed or shared	Rewind (session + code together, seconds)
Change already committed or pushed - team can see it	<code>git revert</code> (auditable, history-preserving)
Damage predates your checkpoints (bad merge, old session)	<code>git</code> - or rebuild from references (Drill 3)

Answer these three in your notes (one line each):

1. Bad refactor, 10 minutes ago, uncommitted -> ?
2. Bad commit pushed to the shared branch an hour ago -> ?
3. Claude installed the wrong package globally -> ?

***Note:** neither tool un-runs side effects. Installed packages, database migrations, and external API calls stay done - rewind restores conversation and Claude's file edits, `git revert` rewrites tracked files only. Anything beyond files needs its own rollback. (Exact restore scope varies by release.)*

Phase 7: Clean Commits (2 min)

Commit these changes as separate commits: one for the refactor, one for the recovery rebuild.

Phase 8: /compact vs /clear (2 min)

/context

Note usage. Then:

/compact

What files did we modify in this lab? What was the last thing we recovered?

Expected output marker: Claude still knows - /compact preserves a summary; /clear wipes everything.

Compact around 50% usage; don't wait until quality degrades deep into the window.

Two controls worth knowing before you leave:

/compact focus on the routing refactor

Summarizing keeps what *you* named instead of what the automatic pass guesses matters. And to change when auto-compaction fires at all:

/autocompact 500k

***The compaction fact that catches people out:** your project-root `CLAUDE.md` and auto memory are **re-injected from disk** after a compact - they always survive. A **nested** `CLAUDE.md` in a subfolder, or any rule with `paths: frontmatter`, is **dropped** until Claude reads a matching file again. Anything that must hold all session belongs in the project-root file.*

FINISH MARKER - Success Checklist

- [] Routes split into src/routes/ modules
- [] Drill 1: deleted file restored via **Rewind code to here**
- [] Drill 2: poisoned context cleared via **Fork conversation from here**
- [] Drill 3: missing module rebuilt from reference files without rewind
- [] All endpoints verified working

- [] Clean conventional commits
- [] You can state the code/conversation/both decision rule
- [] You answered the three rewind-vs-git `revert` scenarios (Phase 6)

If Stuck

Problem	Recovery
Rewind menu won't open	Type <code>/rewind</code> instead
Restored wrong scope	Rewind again - checkpoints remain available
No checkpoint before the damage	That's Drill 3's lesson: rebuild from <code>@</code> reference files
Server stops responding	"The server crashed. Restart it and confirm it responds on port 3100."
Your fix "doesn't work" - old behaviour persists	An orphaned server from an earlier step is still holding port 3100 and serving stale code ; your new one silently failed to bind. Ask: "Something is still listening on port 3100 serving old code. Find that process, stop it, and restart the server."
<code>/compact</code> loses key facts	Re-state them once; consider Compact Instructions in CLAUDE.md (see Workflow Playbook handout)

Debrief Questions

1. When do you rewind code vs conversation vs both?
2. Rewind vs `git revert` - what does each protect you from, and what does neither fix?
3. Which recovery drill maps to the failure you hit most often at work?